Лабораторна робота №2
МАРШРУТИЗАЦІЯ У ФРЕЙМВОРКУ LARAVEL

2.1 Мета та основні завдання роботи

Ознайомитись з правилами маршрутизації у фреймворку Laravel, навчитися створювати методи контролерів, представлення та обробляти дані форм з перевіркою їх коректності.

2.2 Завдання і порядок виконання роботи

Вам слід створити та виконати тестування двох простих проєктів, зокрема створення статичної сторінки «Скажи привіт» з параметром та роботу з формою з валідацією введеної інформації. Завдання виконується в послідовності:

- Ознайомитись з теоретичними відомостями до лабораторної (розд. 2.3).
- Виконати навчальні приклади (кейси) для функціоналу «Говоримо привіт» (case 0 та 1).
- Виконати навчальні приклади (кейси) для роботи з формами (case 2 та 3).
- Розібратися, чому адреси сторінок у Laravel не містять index.php, та перевести всі посилання застосунку на іменовані маршрути (розд. 2.3.2).
- Уточнити індивідуальне завдання.
- Зберегти програмний код виконаних завдань у окремій гілці на GitHub та створити Pull Request.

2.3 Основні теоретичні відомості

2.3.1 Організація проекту

У цій лабораторній роботі задіяно чотири каталоги застосунку:

Каталог	Що в ньому
routes/	оголошення маршрутів, файл web.php
app/Http/Controllers/	контролери — класи, методи яких обробляють запити
app/Http/Requests/	класи правил перевірки даних форм
resources/views/	представлення — шаблони Blade

2.3.2 Правила маршрутизації в Laravel

Маршрут (route) — це правило, яке зв'язує адресу та HTTP-метод запиту з конкретним обробником. Усі маршрути веб-інтерфейсу оголошуються у файлі *routes/web.php*.

Принципова особливість Laravel: маршрут не виводиться з імені контролера і методу, а оголошується явно. У фреймворках з конвенційною маршрутизацією адреса *site/say* автоматично означала виклик методу `actionSay` класу *SiteController*. Тут такого зв'язку немає. Адресу і обробник ви призначаєте самостійно, тому перейменування контролера не ламає адреси сторінок, а одну й ту саму адресу можна передати іншому контролеру. Посилання в шаблонах при цьому виправляти не доведеться.

Найпростіший маршрут повертає відповідь прямо з функції:

```
Route::get('/hello', function () {
    return 'Привіт!';
});
```

Маршрут до методу контролера:

```
use App\Http\Controllers\SiteController;

Route::get('/say', [SiteController::class, 'say']);
```

Параметри маршруту записуються у фігурних дужках і потрапляють до методу контролера як аргументи в тому самому порядку:

```
Route::get('/say/{message}', [SiteController::class, 'say']);
```

Знак питання робить параметр необов'язковим. У цьому разі відповідний аргумент методу обов'язково має значення за замовчуванням:

```
Route::get('/say/{message?}', [SiteController::class, 'say']);
```

Обмеження на параметр задають методом where. Наприклад, щоб ідентифікатор складався лише з цифр:

```
Route::get('/blog/{id}', [BlogController::class, 'show'])
    ->where('id', '[0-9]+');
```

Іменований маршрут отримує ім'я методом name:

```
Route::get('/entry', [SiteController::class, 'entry'])->name('entry.form');
```

Після цього адреса в шаблонах не пишеться вручну, а будується за іменем:

```
<a href="{{ route('entry.form') }}">Форма</a>
```

Користь тут суто практична. Якщо адреса зміниться *з /entry* на */contact-form*, достатньо виправити один рядок у *routes/web.php*, і всі посилання в застосунку почнуть вести на нову адресу. Адреси, написані в шаблонах руками, довелося б шукати і правити по одній.

Крім *Route::get* маршрут можна оголосити методами *post, put, patch, delete і options*. Браузерна форма вміє надсилати лише *GET* і *POST*, тому для решти методів застосовується підміна: у форму додається директива `@method`, а Laravel читає її і підставляє потрібний метод.

```
<form method="POST" action="{{ route('blog.update', $blog) }}">
    @csrf
    @method('PUT')
    ...
</form>
```

Якщо один обробник має відповідати на кілька методів, використовують `Route::match(['get', 'post'], ...)` або `Route::any(...)`.

Коли сторінка нічого не обчислює, контролер для неї зайвий — маршрут віддає шаблон напряму:

```
Route::view('/about', 'site.about')->name('site.about');
```

Цей спосіб стане в пригоді в індивідуальному завданні, де потрібно зробити лендінг.

Групи маршрутів дозволяють винести спільні властивості за дужки. Наприклад, усі адреси адміністративної частини починаються з */admin*, мають спільний префікс в іменах і вимагають входу в систему:

```
Route::prefix('admin')
    ->name('admin.')
    ->middleware('auth')
    ->group(function () {
       Route::get('/users',        [UserController::class,
'index'])->name('users');
    });
```

Маршрут усередині групи отримає адресу */admin/users* та ім'я *admin.users*. Без групи ці три властивості довелося б повторювати в кожному рядку.

На робочому сервері перелік маршрутів компілюють в один файл. Обробка запиту від цього помітно прискорюється:

```
php artisan route:cache
php artisan route:clear
```

Під час розробки кеш не вмикають: після кожної зміни *routes/web.php* його довелося б скидати вручну.

Група маршрутів ресурсу оголошується одним рядком і створює сім маршрутів для типових операцій над сутністю:

```
Route::resource('blog', BlogController::class);
```

Цей спосіб докладно розглядається в лабораторній роботі № 3.

Перелік усіх оголошених маршрутів зручно переглядати командою:

```
php artisan route:list
```

«Гарні» адреси ***(Pretty URLs)*** в Laravel такі за замовчуванням. Усі запити спрямовуються на єдиний фронт-контролер *public/index.php*, а сама назва файлу в адресі не з'являється — за це відповідає файл *public/.htaccess* для Apache або відповідна директива *try_files* для nginx. Під час розробки, коли застосунок піднято командою `php artisan serve`, переписування адрес виконує вбудований сервер, тому нічого налаштовувати не потрібно.

2.3.3 Завантаження представлення — рендеринг

Метод контролера повертає результат виклику функції `view()`. Перший аргумент — ім'я шаблону, де крапка замінює роздільник каталогів, другий — масив даних для підстановки:

```
return view('site.say', ['message' => $message]);
```

Цей виклик виведе на екран шаблон *resources/views/site/say.blade.php*, передавши до нього змінну *$message*.

Шаблонізатор Blade має такі основні директиви:

Директива	Призначення
`@extends('layouts.app')`	вказує загальний шаблон, у який вбудовується сторінка
`@section('name') … @endsection`	наповнює іменовану ділянку загального шаблону
`@yield('name')`	у загальному шаблоні позначає місце вставки секції
`@include('site._partial')`	вставляє інший шаблон у поточне місце
`@if`, `@foreach`, `@forelse`	умови і цикли
`@csrf`	вставляє у форму прихований токен захисту
`@error('field') … @enderror`	виводить повідомлення про помилку валідації поля

Виведення значень має дві форми:

```
{{ $message }}      {{-- екранує спеціальні символи --}}
```

```
{!! $message !!}    {{-- вставляє розмітку як є --}}
```

Перша форма перетворює символи *<, >, &* на безпечні послідовності, тому браузер покаже їх як текст. Друга вставляє вміст у сторінку без обробки. Для даних, які прийшли від користувача, застосовується тільки перша форма: інакше зловмисник зможе вставити у вашу сторінку власний скрипт.

Передати дані до представлення можна трьома рівноцінними способами:

```
return view('site.say', ['message' => $message]);     // масив
return view('site.say', compact('message'));   // з імен змінних
return  view('site.say')->with('message',  $message);  // ланцюжком
```

Функція `compact('message')` збирає масив `['message' => $message]` з однойменної змінної — це найкоротший запис, коли імен кілька.

Повторювані фрагменти розмітки виносять в окремі шаблони і підключають директивою `@include`, якій теж можна передати дані:

```
@include('site._card', ['title' => $blog->title])
```

Для переліків зручна директива `@forelse`, яка поєднує цикл і перевірку на порожній результат:

```
@forelse ($items as $item)
    <li>{{ $item->title }}</li>
@empty
    <li>Записів немає.</li>
@endforelse
```

Окремо від виведення шаблону контролер може повернути переадресацію:

```
return redirect()->route('entry.form');
```

Різниця між ними така: `view()` формує і повертає HTML сторінки за поточною адресою, а `redirect()` не формує нічого: він наказує браузеру перейти на іншу адресу і зробити новий запит.

2.3.4 Моделі та правила валідації даних

У класичному MVC модель відповідає і за дані, і за правила їх перевірки. Laravel розділяє ці дві задачі:

- **модель** (*app/Models*) описує таблицю бази даних і роботу з нею;
- **клас запиту** (*app/Http/Requests*) описує правила, яким мають відповідати дані, що надійшли з форми.

Такий поділ дає змогу мати різні правила перевірки для різних форм над однією і тією самою таблицею — наприклад, при реєстрації пароль обов'язковий, а при редагуванні профілю ні.

Клас запиту створюється командою:

```
php artisan make:request EntryRequest
```

Він має два основні методи:

- `authorize()` відповідає на питання, чи має поточний користувач право надсилати цю форму взагалі.
- `rules()` повертає масив правил для кожного поля:

```
public function rules(): array
{
    return [
        'name'  => ['required', 'string', 'max:255'],
        'email' => ['required', 'email'],
    ];
}
```

Найуживаніші правила валідації:

- `'required'` – поле обов’язкове і не може бути порожнім;
- `'email'` – перевіряє формат електронної адреси;
- `'max:255'` і `'min:3'` — обмеження довжини;

- `'numeric'` та `'integer'` — перевіряє числові значення;
- `'unique:users,email'` — значення не повторюється в указаній таблиці;
- `'confirmed'` — поле має збігатися з полем <ім'я>_confirmation.

Метод `messages()` дозволяє задати власні тексти помилок замість стандартних англомовних:

```
    public function messages(): array
    {
        return [
         'name.required'  =>  'Поле  «Ім\'я»  не  може  бути
порожнім.',
        ];
    }
```

Якщо правило не описується стандартним набором, його задають замиканням. Наприклад, поле не повинно містити службового слова:

```
public function rules(): array
{
    return [
      'name' => [
      'required',
      function  (string  $attribute,  mixed  $value,  \Closure
$fail) {
                if         (str_contains(mb_strtolower($value),
'admin')) {
                   $fail('Це ім\'я використовувати не можна.');
                }
           },
      ],
    ];
}
```

Дані форми доступні через об'єкт Request: `$request->input('name')` повертає значення поля, `$request->query('page')` —

параметр із рядка адреси, `$request->has('email')` перевіряє наявність поля, `$request->only(['name', 'email'])` повертає лише перелічені поля. Клас запиту *EntryRequest* успадковує всі ці методи, тому в контролері доступні і вони, і результат перевірки.

Щоб правила запрацювали, клас запиту достатньо вказати як тип аргументу методу контролера. Перевірку буде виконано ще до того, як почне виконуватися тіло методу:

```
public function entryStore(EntryRequest $request)
{
    return view('site.entry-confirm', $request->validated());
}
```

Якщо дані не пройшли перевірку, Laravel самостійно поверне користувача на попередню сторінку, передавши туди перелік помилок і раніше введені значення. У шаблоні вони доступні так:

```
<input type="text" name="name" value="{{ old('name') }}">
@error('name') <p class="error">{{ $message }}</p> @enderror
```

Якщо потрібно показати всі помилки одним переліком угорі сторінки, використовують змінну `$errors`, яка доступна в кожному представленні:

```
@if ($errors->any())
    <div class="errors">
        <ul>
            @foreach ($errors->all() as $error)
                <li>{{ $error }}</li>
            @endforeach
        </ul>
    </div>
@endif
```

Обидва способи можна поєднувати: перелік угорі дає загальну картину, а `@error` показує проблему безпосередньо біля поля.

Функція `old('name')` повертає те, що користувач ввів минулого разу, тому форма не очищується після помилки. Метод `validated()` повертає лише ті поля, які описані в правилах — випадково надіслані зайві поля до застосунку не потраплять.

2.4 Навчальні приклади

У цьому розділі розглянуто базові принципи побудови веб-сторінок у фреймворку Laravel: створення маршрутів, контролерів і представлень, передавання даних між компонентами застосунку, оброблення HTTP-запитів, створення HTML-форм та валідація введених користувачем даних.

Навчальні приклади побудовані послідовно. Спочатку буде створено просту сторінку, яка отримує параметр із URL та відображає його користувачу. Після цього буде розглянуто використання представлень Blade та спільного макета сторінок. У наступних прикладах буде створено форму введення даних, реалізовано її перевірку та оброблення POST-запиту.

2.4.1 Case0 — створення маршруту та методу контролера

У цьому розділі описано, як створити нову статичну сторінку "Говоримо привіт". Для цього слід створити дію (*action*) в контролері та представлення (*view*).

У Laravel маршрути, призначені для звичайних веб-сторінок, зазвичай описуються у файлі *routes/web.php.*

Проте перед створенням маршруту необхідно створити контролер, у якому буде розташований метод для оброблення запиту. Перебуваючи в кореневому каталозі Laravel-проєкту, виконайте команду:

```
php artisan make:controller SiteController
```

Команда `make:controller` є частиною консольного інструмента Laravel Artisan. Вона автоматично створює клас контролера та розміщує його у стандартному каталозі: *app/Http/Controllers/SiteController.php*.

Після виконання команди відкрийте створений файл. Базовий контролер матиме приблизно такий вигляд:

```
<?php

namespace App\Http\Controllers;

class SiteController extends Controller
{}
```

Додайте до класу метод ***say()***:

```
<?php

namespace App\Http\Controllers;

class SiteController extends Controller
{
    public function say(string $message = 'Привіт')
    {
        return view('site.say', ['message' => $message]);
    }
}
```

У нашому випадку цей метод приймає один параметр `$message`. Для цього параметра вказано значення за замовчуванням: `'Привіт'`. Тому, якщо користувач не передасть повідомлення в URL, на сторінці буде відображено слово *«Привіт»*.

Функція `view()` завантажує представлення та передає до нього необхідні дані. Запис `view('site.say')` означає, що Laravel буде шукати файл: *resources/views/site/say.blade.php*. Змінна `$message`, отримана методом контролера, передається до цього представлення та стає доступною всередині Blade-шаблону.

Тепер необхідно зв'язати URL із створеним методом контролера. Відкрийте файл: *routes/web.php* та додайте імпорт контролера:

```
use App\Http\Controllers\SiteController;
```

Після цього оголосіть маршрут:

```
Route::get(
    '/say/{message?}',
    [SiteController::class, 'say']
)->name('site.say');
```

Розглянемо цей запис детальніше.

Маршрут обробляє HTTP-запити типу ***GET.*** Адреса */say/{message?}* містить параметр маршруту *message*. Фігурні дужки `{}` позначають динамічний параметр URL. А знак питання `?` в кінці показує, що параметр є необов'язковим.

Отже, обидві адреси будуть правильними: http://127.0.0.1:8000/say та http://127.0.0.1:8000/say/Вітаю. У першому випадку параметр не передається, тому Laravel використає значення за замовчуванням: *«Привіт»,* а у другому випадку значення: *«Вітаю»* буде передано до параметра `$message` методу `say()`.

Частина `[SiteController::class, 'say']` визначає, який саме код має бути виконаний. Laravel знаходить клас SiteController та викликає його метод say(). Таким чином, маршрут встановлює зв'язок між URL-адресою та програмною логікою застосунку.

Метод `->name('site.say')` задає маршруту ім'я. Іменовані маршрути дозволяють надалі створювати посилання без жорсткого записування URL. Наприклад: `route('site.say')`.

Якщо в майбутньому адреса маршруту зміниться, достатньо буде змінити її лише у файлі маршрутів.

Після створення маршруту перевірте, чи Laravel його зареєстрував. Для цього виконайте команду:

```
php artisan route:list
```

У переліку маршрутів має з'явитися запис приблизно такого вигляду:

```
GET|HEAD  say/{message?}  site.say  SiteController@say
```

Команда `route:list` є корисним засобом перевірки маршрутизації. Вона показує HTTP-метод, URL, ім'я маршруту та контролер, який його обробляє.

На цьому етапі маршрут і метод контролера створено, однак представлення site.say ще відсутнє. Його буде створено у наступному прикладі.

2.4.2 Case1 — створення представлення

Представлення (***view***) відповідає за формування HTML-коду сторінки, який надсилається браузеру користувача.

У Laravel для створення представлень використовується шаблонізатор *Blade*. Він дозволяє поєднувати звичайний HTML-код із даними, отриманими з PHP, а також використовувати спеціальні директиви Laravel.

Створіть каталог *resources/views/site*, якщо він ще не існує. У ньому створіть файл *resources/views/site/say.blade.php* та додайте до нього такий код:

```
@extends('layouts.app')
```

Ця директива означає, що це представлення використовує спільний макет сторінки *resources/views/layouts/app.blade.php*.

Такий підхід дозволяє винести загальні елементи сайту — наприклад, меню, заголовок сторінки, підвал або підключення CSS — до одного файлу. У результаті кожна окрема сторінка містить лише власний унікальний контент.

```
@section('title', 'Говоримо привіт')
```

Ця директива передає значення до секції title, яка може використовуватися в основному макеті для формування заголовка HTML-сторінки.

```
@section('content')
```

Ця конструкція визначає початок секції основного вмісту сторінки, а її завершення позначається директивою:

`@endsection`.

Всередині секції запишемо наступний код, щоб зчитати дані з контролера:

```
<h1>{{ $message }}</h1>
```

Важливо, що Blade автоматично екранує HTML-символи та вивоить значення PHP-змінної при використанні конструкції `{{ }}`. Це захищає сторінку від виведення небезпечного HTML або JavaScript-коду, отриманого від користувача.

Тепер запустіть Laravel-застосунок, якщо сервер ще не працює. Відкрийте http://127.0.0.1:8000/say, на сторінці має відобразитися «Привіт». Це підтверджує, що значення параметра `$message`, задане у методі контролера за замовчуванням.

Після цього відкрийте http://127.0.0.1:8000/say/Вітаю. Тепер на сторінці має відобразитися «Вітаю» (рис. 2.1)

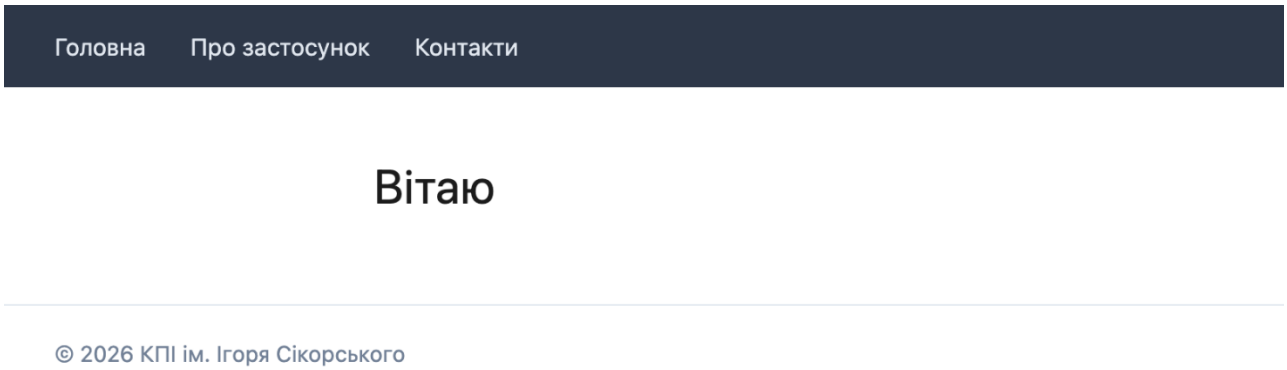


Рисунок 2.1 – «Говоримо Вітаю». Результат виконання маршруту, методу контролера та представлення

👁 **Зверніть увагу**: сторінка містить спільну шапку, меню та футер, хоча у файлі *say.blade.php* ці елементи відсутні. Це результат використання `@extends('layouts.app')`, Laravel спочатку обробляє дочірнє представлення, а потім вставляє секцію *content* у відповідне місце загального макета.

Такий принцип дозволяє уникати дублювання HTML-коду та є типовим для сучасної веб-розробки.

2.4.3 Case2 — створення класу запиту та правил валідації даних

У попередніх прикладах дані передавалися безпосередньо через URL. А у реальних веб-застосунках значна частина інформації надходить від користувача через HTML-форми. Наприклад, це можуть бути:

- ім'я;
- електронна пошта;
- пароль;
- текст повідомлення;
- номер телефону;
- параметри пошуку.

Дані, отримані від користувача, не можна вважати правильними або безпечними без попередньої перевірки. Тому одним із важливих етапів оброблення HTTP-запиту є валідація даних.

Валідація — це перевірка, чи відповідають отримані дані встановленим правилам.

У цьому прикладі потрібно створити форму з двома полями: *name* та *email*. Для них будуть встановлені такі правила:

1. поле name є обов'язковим;
2. поле name повинно містити рядок;
3. довжина імені не повинна перевищувати 255 символів;
4. поле email є обов'язковим;
5. значення email повинно мати формат адреси електронної пошти.

Laravel дозволяє перевіряти дані безпосередньо у контролері. Проте для більш структурованого коду правила доцільно винести в окремий клас ***Form Request***.

Створіть такий клас командою:

```
php artisan make:request EntryRequest
```

В результаті Laravel створить файл *app/Http/Requests/EntryRequest.php*. Відкрийте його та додайте наступні методи:

```
public function authorize(): bool
{
    return true;
}
```

Цей метод визначає, чи має користувач право виконувати цей запит.

У нашому навчальному прикладі авторизація ще не використовується, тому метод повертає просто true. Це означає, що запит дозволено для всіх користувачів.

Для створення правил перевірки вхідних даних користувача, використовується наступний метод:

```
public function rules(): array
  {
      return [
          'name' => ['required', 'string', 'max:255'],
          'email' => ['required', 'email'],
      ];
  }
```

Для поля *name* встановлено три правила:

- `'required'` означає, що поле не може бути порожнім.
- `'string'` перевіряє, що значення є рядком.
- `'max:255'`обмежує максимальну довжину значення.

Поле *email* також є обов'язковим, а правило `'email'` перевіряє формат введеної адреси.

Метод `messages()`дозволяє перевизначити стандартні повідомлення Laravel про помилки валідації.

```
public function messages(): array
{
```

```
    return [
            'name.required'  =>  'Поле «Ім\'я» не може бути
порожнім.',
            'email.required'  =>  'Поле «Email» не може бути
порожнім.',
            'email.email' => 'Email введено некоректно.',
        ];
    }
}
```

Важливо розуміти, що серверна валідація повинна виконуватися незалежно від того, чи використовується додаткова перевірка в браузері.

HTML та JavaScript можуть покращити зручність для користувача, проте клієнтські перевірки можна обійти. Саме тому остаточна перевірка отриманих даних завжди повинна виконуватися на сервері.

У вихідному варіанті цього прикладу також наголошувалося, що клієнтська перевірка покращує користувацький досвід, але серверна перевірка є обов'язковою.

2.4.4 Case3 — обробка запитів та форми

Після створення правил валідації необхідно реалізувати саму форму та обробити дані, які вона надсилає.

Для цього будуть використані два HTTP-запити:

- *GET /entry* — відображає форму;
- *POST /entry* — приймає та обробляє введені дані.

Це типовий підхід у веб-застосунках. GET-запит використовується для отримання сторінки або даних, а POST-запит використовується для надсилання даних на сервер.

Додайте у *SiteController* імпорт реквесту з правилами валідації:

```
use App\Http\Requests\EntryRequest;
```

Після цього створіть у контролері два методи. Перший метод не отримує та не змінює дані. Він буде тільки повертати сторінку з HTML-формою:

```
public function entry()
{
    return view('site.entry');
}
```

Другий метод — `entryStore()` буде отримувати на вхід об'єкт `EntryRequest`. Отже, перед виконанням цього методу Laravel автоматично застосує наші визначені правила.

```
public function entryStore(EntryRequest $request)
{
     return view(
          'site.entry-confirm',
          $request->validated()
     );
}
```

Якщо дані некоректні, метод контролера взагалі не буде виконано. Laravel автоматично поверне користувача до попередньої сторінки та передасть повідомлення про помилки.

Якщо всі дані пройшли валідацію, код поверне лише ті поля, які були описані правилами валідації. У нашому випадку це *name* та *email*.

Таким чином, до представлення не передаються сторонні поля, навіть якщо користувач спробував штучно додати їх до HTTP-запиту.

Наступний крок, оголосіть маршрути у файлі routes/web.php:

```
Route::get(
    '/entry',
    [SiteController::class, 'entry']
)->name('entry.form');

Route::post(
```

```
    '/entry',
    [SiteController::class, 'entryStore']
)->name('entry.store');
```

👁 **Зверніть увагу**: обидва маршрути мають однакову адресу: */entry*, але різні HTTP-методи. Саме вони і дозволяють Laravel визначити, яку операцію необхідно виконати.

При відкритті адреси в браузері буде виконано *GET /entry* і викликано функцію `entry()`, а після надсилання форми браузер виконає *POST /entry* і Laravel викличе метод `entryStore()`.

Далі створіть представлення форми *resources/views/site/entry.blade.php* і додайте туди наступні елементи:

```
@extends('layouts.app')

@section('title', 'Форма введення даних')

@section('content')
    <h1>Форма введення даних</h1>
    <form method="POST" action="{{ route('entry.store') }}">
```

Ця запис означає, що дані надсилатимуться HTTP-методом POST на адресу, яка визначається за допомогою *action*.

Функція `route()` генерує URL за ім'ям маршруту. Це краще, ніж записувати адресу */entry* безпосередньо, оскільки маршрут надалі можна змінити без необхідності змінювати всі представлення.

Далі в середині звичайних POST-форм Laravel обов'язково має знаходиться директива:

```
@csrf
```

CSRF означає ***Cross-Site Request Forgery*** — підроблення міжсайтового запиту. Для захисту від цього Laravel додає до форми спеціальний прихований токен: `<input type="hidden" ...>`. Під час

надсилання форми сервер перевіряє цей токен. Якщо він відсутній або неправильний, Laravel відхиляє запит. Тому, якщо видалити `@csrf`, POST-запит, як правило, завершиться помилкою зі статусом 419.

Наступним кроком додаємо поле форми для введення імені.

```
<div>
        <label for="name">Ім'я</label>

        <input
            type="text"
            id="name"
            name="name"
            value="{{ old('name') }}"
        >
```

Якщо при відправці даних валідація не пройшла, Laravel повертає користувача назад до форми, але завдяки функції `old()` зберігає попередньо введені значення. Отже, користувачу не потрібно вводити всі дані повторно.

Наприклад, якщо ім'я введено правильно, а email — неправильно, поле імені після перезавантаження сторінки залишиться заповненим.

Додамо фрагмент коду для виведення помилок валідації:

```
    @error('name')
            <p class="error">{{ $message }}</p>
        @enderror
    </div>
```

Ця конструкція перевіряє, чи існує помилка для поля *name*. Якщо така помилка є, змінна `$message` містить текст повідомлення, сформований валідатором.

Тепер аналогічно треба додати код для поля *email*.

```
<div>
        <label for="email">Email</label>

        <input
```

```
                type="text"
                id="email"
                name="email"
                value="{{ old('email') }}"
            >

            @error('email')
                <p class="error">{{ $message }}</p>
            @enderror
        </div>

        <button type="submit">Надіслати</button>
    </form>
@endsection
```

Після відправлення (*submit*) форми, необхідно створити сторінку для підтвердження цього, наприклад: *resources/views/site/entry-confirm.blade.php*.

```
@extends('layouts.app')

@section('title', 'Дані прийнято')

@section('content')
    <h1>Дані прийнято</h1>

    <p>Ви ввели такі дані:</p>

    <ul>
        <li>
            <strong>Ім'я:</strong>
            {{ $name }}
        </li>

        <li>
            <strong>Email:</strong>
```

```
                {{ $email }}
            </li>
        </ul>
        <p>
            <a href="{{ route('entry.form') }}">
                Повернутися до форми
            </a>
        </p>
    @endsection
```

> **Примітка:** Після успішної валідації метод `$request->validated()` поверне масив приблизно такого вигляду:
>
> [
>
> 'name' => 'Іван',
>
> 'email' => 'ivan@example.com',
>
>]
>
> Цей масив у контролері передається безпосередньо до представлення:
>
> ```
> return view(
>     'site.entry-confirm',
>     $request->validated()
> );
> ```
>
> Саме тому у Blade стають доступними змінні `$name` та `$email`.

Перевіримо роботу форми. Для цього у браузері відкрийте адресу http://127.0.0.1:8000/entry і має з'явитися форма з двома полями: *Ім'я* та *Email*.

Спочатку натисніть кнопку «Надіслати» не заповнюючи жодного поля. Оскільки обидва поля мають правило «required», Laravel не дозволить продовжити оброблення запиту та поверне користувача назад до форми, а під полями мають відобразитися повідомлення про помилки (рис. 2.2).

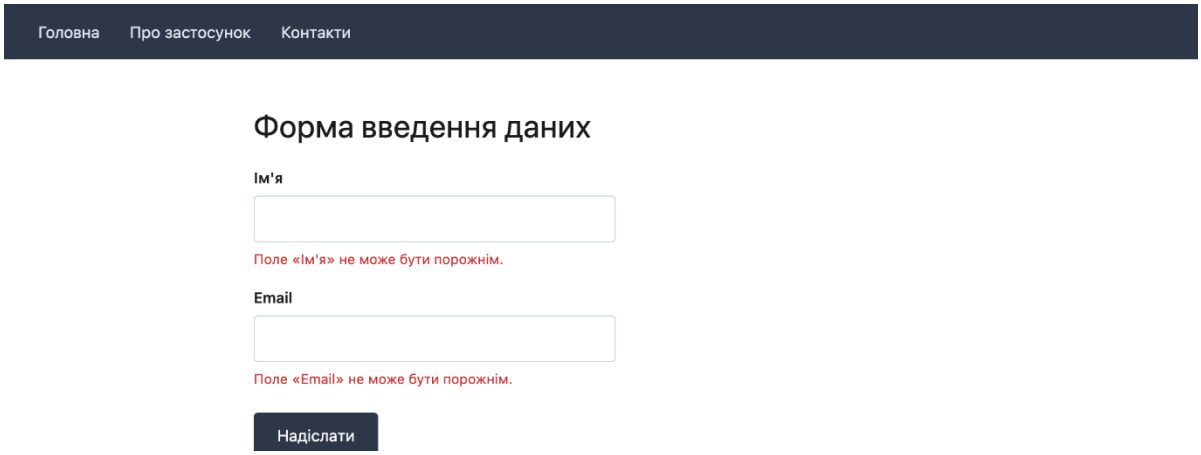


Рисунок 2.2 — Помилки валідації форми

Тепер введіть правильне ім'я, наприклад, Іван, але в поле *Email* введіть: test. Поле імені має залишитися заповненим завдяки: `old('name')`, а під полем *email* має з'явитися повідомлення: *«Email введено некоректно»*.

Після цього введіть правильну адресу, наприклад, petrenko@kpi.ua та повторно натисніть кнопку «Надіслати». Якщо всі значення відповідають правилам валідації, буде виконано метод `entryStore()` і користувач побачить сторінку підтвердження з введеними даними (рис. 2.3).

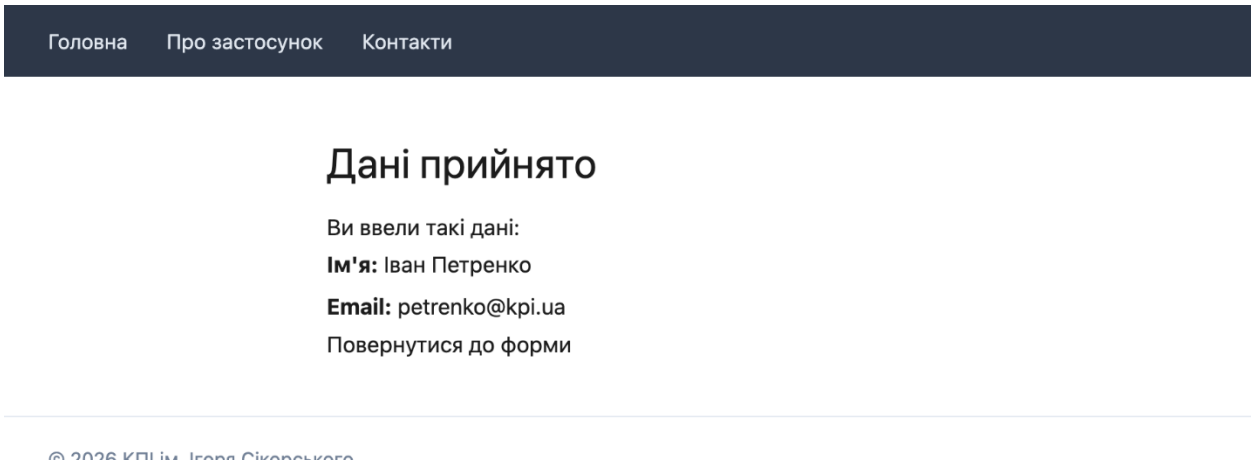


Рисунок 2.3 – Результат успішного оброблення форми

Повний процес оброблення форми зображено на рисунку 2.4. Подібний принцип використовується в більшості реальних веб-застосунків: дані надходять через HTTP-запит, перевіряються на сервері, після чого передаються до бізнес-логіки або зберігаються в базі даних.

2.4.4 Case4 — створення простих статичних сторінок

Наприкінці лабораторної необхідно завершити роботу з головним меню застосунку. Пункти *«Про застосунок»* та *«Контакти»* можуть поки що не мати відповідних сторінок.

Оскільки ці сторінки не містять додаткової програмної логіки і лише відображають HTML-контент, створювати для них окремі методи контролера необов'язково. Laravel дозволяє безпосередньо зв'язати URL із представленням за допомогою `Route::view()`.

Додайте до *routes/web.php* наступне:

```
Route::view(
    '/about',
    'site.about'
)->name('site.about');

Route::view(
    '/contact',
    'site.contact'
)->name('site.contact');
```

Метод `Route::view()` доцільно використовувати для простих статичних сторінок, якщо отримання запиту не потребує виконання додаткової логіки в контролері.

Створіть blade шаблони для них, наприклад:

resources/views/site/about.blade.php

```
@extends('layouts.app')
@section('title', 'Про застосунок')
@section('content')
    <h1>Про застосунок</h1>

    <p>
        Цей навчальний веб-застосунок створено
        під час виконання лабораторних робіт
```

```
            з дисципліни «Програмування на скриптових мовах».
        </p>
    @endsection
```

Для *resources/views/site/contact.blade.php:*

```
    @extends('layouts.app')

    @section('title', 'Контакти')
    @section('content')
        <h1>Контакти</h1>
        <p>
         На цій сторінці може бути розміщена інформація про
         кафедру, викладача або контактні дані навчального
проєкту.
        </p>
    @endsection
```

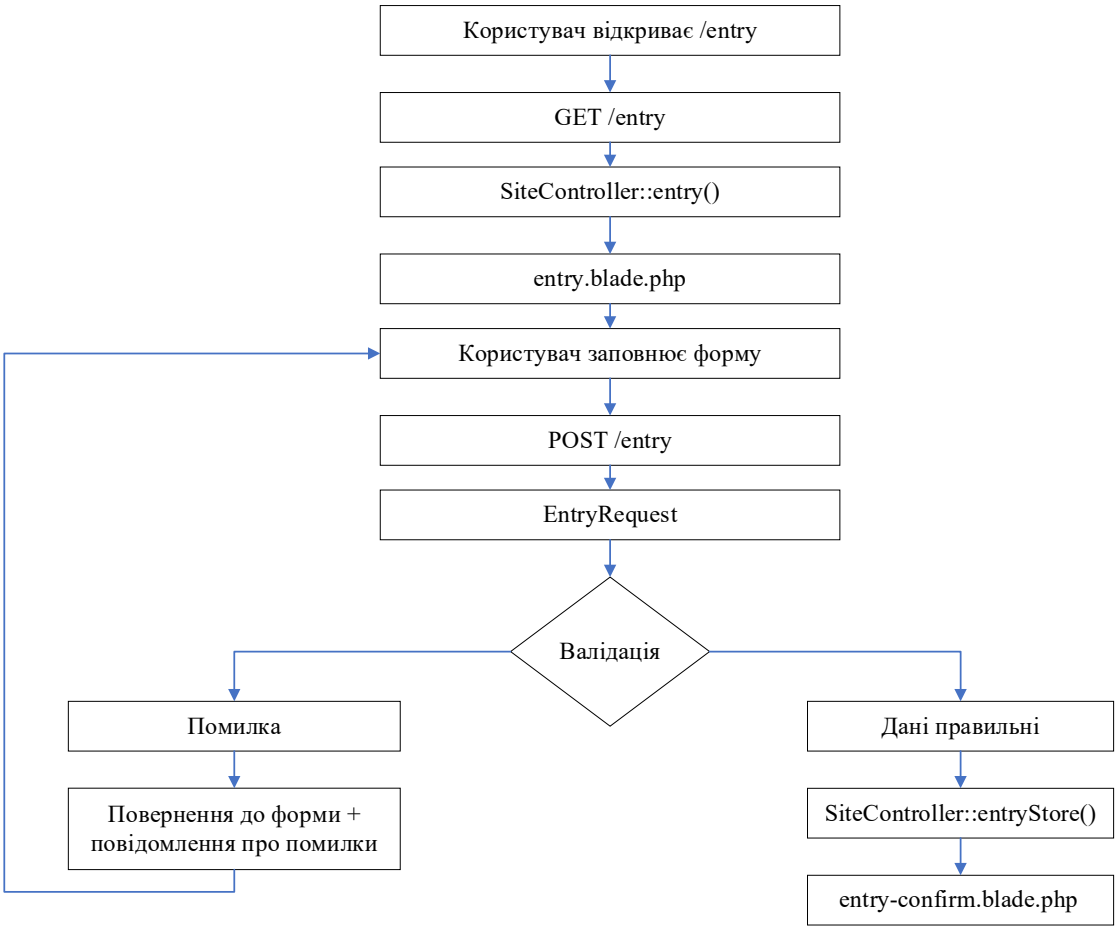


Рисунок 2.4 — Повний процес оброблення форми користувача з Case 3

Самостійно перевірте роботу всіх пунктів меню, жоден з них під час навігації не повинен повертати помилку *404*.

2.5 Індивідуальне завдання

Продовжіть розробку веб-застосунку, розпочатого у лабораторній №1. Створіть окрему інформаційну сторінку або невеликий лендінг, який відповідає предметній області вашого проєкту. Наприклад:

- для *магазину* — сторінка магазину або окремого сервісу;
- для *фітнес-центру* — презентація послуг;
- для *університетської системи* — сторінка факультету або кафедри;

Вам необхідно:

1) створити окремий маршрут для лендінгу;
2) створити метод контролера, який повертає представлення;
3) створити Blade-представлення сторінки;
4) використати загальний макет layouts.app;
5) передати щонайменше одне значення з контролера до представлення;
6) додати форму зворотного зв'язку відповідно до тематики проєкту;
7) форма повинна містити щонайменше: *ім'я користувача, email* та *текст повідомлення*;
8) створити окремий *FormRequest* для перевірки введених даних;
9) реалізувати перевірку обов'язкових полів та формату email;
10) відображати повідомлення про помилки валідації біля відповідних полів;
11) забезпечити збереження раніше введених значень у разі помилки валідації за допомогою функції `old();`
12) після успішного надсилання форми відобразити сторінку підтвердження.

> 💬 **Примітка:** Форма може мати більш прикладне призначення відповідно до предметної області. Наприклад: *«Залишити заявку на*

товар», «Записатися на тренування», «Поставити запитання» або «*Надіслати заявку на участь*».

***Додатково**: реалізуйте надсилання введених контактних даних на електронну пошту засобами Laravel.

Для цього необхідно ознайомитися з механізмом відправлення електронної пошти у Laravel використовуючи офіційну документацію.

Після завершення роботи перевірте:

- відкриття створеної сторінки та роботу всіх маршрутів;
- відображення значень, переданих із контролера;
- реакцію форми на порожні та неправильно заповнені поля;
- збереження правильних значень після помилки валідації;
- успішне оброблення коректно заповненої форми;
- перехід на сторінку підтвердження;
- надсилання контактних даних на email.

За результатами виконання індивідуального завдання підготуйте звіт із детальним описом ходу роботи. Зберегти виконані зміни у новій гілці на GitHub, наприклад, `lab2` та створити Pull Request до гілки `main`.

Контрольні питання

1. Що таке маршрутизація у фреймворку Laravel і яку функцію вона виконує?
2. Як Laravel визначає, який контролер і метод викликати при зверненні до певного URL?
3. Яке призначення файлу *routes/web.php* у структурі застосунку?
4. Чому адреси сторінок у Laravel не містять *index.php* і яку роль у цьому відіграють фронт-контролер та переписування адрес веб-сервером?

5. Поясніть структуру оголошення `Route::get('/say/{message?}', [SiteController::class, 'say'])` — які його частини відповідають адресі, контролеру, методу та параметру?

6. Як у Laravel оголосити власний маршрут з параметром, накласти на цей параметр обмеження та надати маршруту ім'я?

7. Яка різниця між `view()`, директивою `@include` та `redirect()` у контролерах і шаблонах Laravel?

8. Яким чином передаються дані з контролера до представлення?

9. Що таке модель у архітектурі MVC та яку роль вона виконує в Laravel?

10. Як у Laravel задаються правила валідації введених користувачем даних?

11. Яким чином можна перевірити коректність даних, отриманих через форму, перед збереженням у базу даних?

12. Що відбувається у фреймворку Laravel від моменту надходження HTTP-запиту до формування відповіді користувачеві (життєвий цикл запиту)?

Література до лабораторної роботи № 2

1. Laravel Documentation. Routing. – https://laravel.com/docs/12.x/routing
2. Laravel Documentation. Blade Templates. – https://laravel.com/docs/12.x/blade

Laravel Documentation. Validation. – https://laravel.com/docs/12.x/validation